Sparse Attention Windows for Summarizing Long Technical Reports

Anonymous Authors
Affiliation withheld for review

TABLE I
SUMMARY QUALITY ON HELD-OUT REPORTS

Model	ROUGE-1	ROUGE-L
Truncate to 1024 tokens	38.2	34.0
Sliding window only	39.5	35.1
Window + headings (ours)	40.4	36.3

***Abstract*—Transformer summarizers handle a few thousand tokens, but technical reports run to tens of thousands. We restrict attention to a sliding window plus a small set of section-heading tokens that every token can see. Memory then grows linearly with length, and on a set of 2,000 engineering reports the summaries score 2.3 ROUGE-L points above a model that truncates the input.**

***Index Terms*—summarization, transformers, sparse attention, long documents**

I. Introduction

Self-attention compares every token with every other token [1], so memory grows with the square of the input length. Pretrained encoders such as BERT [2] therefore stop at 512 tokens, and summarizers built on them read only the start of a long report. In engineering reports the findings are often near the end.

II. Method

Each token attends to the 256 tokens on either side and to every section-heading token in the document. Heading tokens attend to everything. This keeps the report's structure visible from any position at a cost that grows linearly with length. The decoder is unchanged. We fine-tuned for 40,000 steps with Adam [3] and dropout [4] of 0.1 on 2,000 reports with author-written executive summaries.

III. Results

Table I shows that the heading tokens add more than a plain sliding window. Summaries written from the full report mention the conclusions section three times as often as truncated ones.

IV. Conclusion

Letting every token see the section headings is a cheap way to give a sparse-attention summarizer the structure of a long report. It needs no change to the decoder and fits on a single GPU for reports of 32,000 tokens.

References

[1] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.
[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.